학습 노트 (7). Q.14 레이아웃과 4 차 MPW 설계 계획

오픈소스 반도체 설계 및 제작 종합설계 (2026 Fall)

공과대학 기계공학부 안준영　학번 2024-11078　2026 년 11 월 12 일

들어가며

이번 주에는 우주 침략자 예제 Q.14 로 과제 15~19 를 했다. 인공지능이 낸 RTL 을 시뮬레이션하고, 표준 셀로 합성해 배치와 배선을 하고, 적층 비아를 없앤 레이아웃을 만들었다. 수업에서 정한 표준 칩의 설계 영역은 1000 × 1000 µm 인데, 예제 RTL 은 배치 밀도를 어떻게 바꿔도 이 안에 들어가지 않았다. 그래서 게임 동작은 그대로 두고 회로 구조만 바꿔 936.6 × 913.05 µm 로 줄였다. 바꾼 RTL 이 원래 RTL 과 같은 동작을 하는지는 매 클럭 출력을 비교해서 확인했다. 마지막으로 PSCE 키트에서 에뮬레이션과 프로토타이핑을 돌렸다. 여기까지가 1 부다. 2 부는 4 차 MPW 에 낼 내 칩 bb_node 의 설계 계획서다.

1 부. 우주 침략자 Q.14 (과제 15~19)

1. 실습 환경

합성과 레이아웃은 x86 리눅스에서 했다. yosys 0.58, qflow 1.4.104, qrouter 1.4.90, magic 8.3, netgen 1.5, Verilator 5.030, SystemC 3.0.2 를 썼다. 수업 VM 과 달라서 생긴 차이가 세 가지 있다. 첫째, 설치된 qflow 스크립트 16 개의 첫 줄이 #!tclsh 로 되어 있어 작업 폴더에 tclsh 가 없으면 실행되지 않았다. 이 때문에 qflow 가 전원 이름을 vdd 가 아니라 VDD 로 잡았고, 전원 스트라이프가 생기지 않아 배선이 175 개 넘게 실패했다. 첫 줄을 /usr/local/bin/tclsh 로 고친 뒤에야 키트 기본 설정의 크기가 1479.6 × 985.05 µm 로 블로그(1461.6 × 985.05)와 거의 같게 나왔다. 둘째, qrouter 를 Tcl 지원으로 다시 빌드해 수업 설치와 같은 스크립트 배선 모드를 쓰게 했다. 셋째, yosys 버전이 달라 합성 결과가 1,909 셀(LVS 기준)로 블로그(1,838)와 조금 다르다.

키트 실행은 PSCE-TRANS(Cyclone IV EP4CE6)와 PSCE-MI(라즈베리 파이 피코)를 연결한 리눅스 데스크톱에서 했다. FPGA 이미지는 Quartus 25.1std Lite 로, 피코 펌웨어는 arduino-cli(arduino-pico 6.3.0)로 만들었다. JTAG 케이블은 Makefile 에 적힌 USB-Blaster 가 아니라 FT232H 여서 openFPGALoader 의 케이블 이름을 digilent_hs2 로 바꿔 썼다.

2. 과제 15: 인공지능이 낸 RTL 의 시뮬레이션

15-2 절의 대화 기록(Talks)에는 인공지능이 같은 요청에 RTL 을 세 번 냈다고 되어 있다. 수업 Makefile 의 lint 로 네 파일을 검사한 결과는 TABLE 1 과 같다. v_1 과 v_2 의 오류는 대화 기록에서 질문자가 지적한 내용과 같았다.

TABLE 1. Q.14 RTL 의 lint 결과 (수업 Makefile 설정)

파일	결과	원인
1 No_Score (수동 최적화)	통과	
2 v_1	오류 3 개	c, t_invader_x, t_invader_y 를 선언하지 않고 씀
3 v_2	SELRANGE 2, INFINITELOOP 1	3 비트 loop_r 에서 0-1 이 7 이 되어 loop_r >= 0 이 항상 참
4 v_3	통과	

게임은 키보드로 해야 해서 손으로는 같은 조건을 반복할 수 없다. 그래서 키 입력을 정해진 순서로 넣고 화면을 프레임마다 파일로 남기는 무인 테스트벤치를 따로 만들었다. DUT 연결과 화면 복원 방식은 수업 테스트벤치의 sc_glcd128x64_TLM 과 같다. 10 프레임에 발사로 게임을 시작하고, 40 프레임마다 발사하고, 좌우 이동을 반복하는 입력으로 2,400 프레임을 돌렸다. v_3 과 No_Score 의 화면은 2,400 프레임 모두 화소 단위로 같았고, 격추, 피격, GAME OVER, 재시작이 모두 나왔다(Fig. 1).



Fig. 1. v_3 을 무인 테스트벤치로 돌린 화면. 시뮬레이션이 프레임마다 남긴 파일을 그대로 늘렸다.

화면을 보다가 두 RTL 에 똑같이 있는 문제 두 가지를 찾았다. 하나는 목숨 표시다. 처음 목숨은 3 인데 아이콘이 두 개만 나온다. 세 번째 아이콘의 조건 scan_x < 7'd128 에서 7 비트로 128 을 쓰면 0 이 되어 조건이 항상 거짓이다. 수업 Makefile 은 -Wno-UNSIGNED -Wno-WIDTHTRUNC 로 이 경고를 끄는데, verilator -Wall 로 보면 "Comparison is constant due to unsigned arithmetic"으로 나온다. 다른 하나는 화면이 오른쪽으로 1 화소 밀리는 것이다. GAME OVER 테두리의 오른쪽 세로선이 왼쪽 끝에 붙어 나온다. v_sync 는 조합 출력이고 lcd_data 는 레지스터 출력이라 둘이 1 클럭 어긋나기 때문이다.

인공지능의 면적 평가도 확인했다. 대화 기록에서 v_3 은 기존 구조보다 면적을 80% 이상 줄였고 1,800~2,200 게이트로 1.0 mm² 의 8~10%를 쓴다고 했다. 같은 합성 스크립트로 합성하니 v_3 은 1,810 셀, 셀 실면적(LEF 에 적힌 셀 크기의 합) 1.093 mm² 로 No_Score(1,829 셀, 1.083 mm²)보다 오히려 0.9% 컸다. v_3 이 바꾼 것은 총알과 외계인의 충돌 검사를 25 개 병렬에서 한 클럭에 하나씩으로 바꾼 것이다. 그런데 그 대신 인덱스 계산과 상태 레지스터 다섯 개가 늘었고, 면적의 대부분을 차지하는 그리기 회로는 그대로였다.

3. 과제 18: 면적 최소화

입력 핀 다섯 개(clk, rst_n, btn_left, btn_right, btn_fire)는 위, 출력 핀 세 개(clk_o, v_sync, lcd_data)는 아래로 .cel2 에 고정했다. 그다음 배치 밀도와 가로세로비를 바꿔 가며 배치, 배선, LVS, 크기 측정을 15 번 돌렸다. 대표 결과는 TABLE 2 와 같다. 배선이 성공하는 가장 작은 면적은 밀도 0.82, 가로세로비 0.8 의 1344.6 × 1057.05 µm(1.421 mm^2)였고, 밀도 0.83 부터는 배선이 실패했다. 가로세로비는 모양만 바꾸고 면적에는 거의 영향이 없었다.

TABLE 2. 배치 밀도와 가로세로비에 따른 크기 (No_Score, 입력 위·출력 아래)

밀도	가로세로비	크기 [µm]	면적 [mm^2]	배선 실패
0.80	0.7	1479.6 × 985.05	1.458	0
0.80	1.0	1212.6 × 1201.05	1.456	5
0.82	0.8	1344.6 × 1057.05	1.421	0
0.83	0.7	1425.6 × 985.05	1.404	15
0.85	0.7	1392.6 × 985.05	1.372	12
0.90	0.7	1365.6 × 949.05	1.296	43

1000 × 1000 µm 는 배치 옵션으로는 맞출 수 없다. 배치된 셀 1,909 개의 실면적만 1.126 mm^2 라서 밀도를 100%로 올려도 1.0 mm^2 를 넘는다. 합성 보고서의 면적만 보면 이것을 놓친다. Liberty 파일의 area 값이 실제 셀 크기보다 작기 때문이다. 예를 들어 DFFSR 은 Liberty 에 1,584 μm^2 로 적혀 있지만 LEF 의 셀 크기는 2,592 μm^2 다.

4. 1000 × 1000 µm 에 맞추기

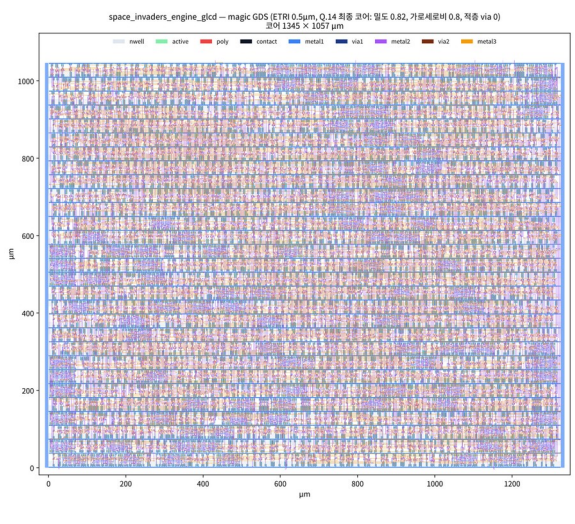
면적이 어디에 쓰이는지 알아보려고 기능을 하나씩 빼고 합성해 봤다. 플립플롭 102 개가 면적의 25%였고, 외계인 미사일 발사 위치 찾기가 14%, 외계인 그리기가 10%였다. 이것을 보고 게임 동작은 그대로 두고 다음 네 가지를 바꿨다.

- 외계인 미사일 발사. 발사 검사는 game_tick 일 때만 하는데, 그때 game_tick_counter 는 항상 32767 이다. 그래서 rand_counter[3:0]==4'hF 는 언제나 참이고 shoot_col 은 언제나 2 다. 무작위로 보이던 열 선택이 사실은 고정이었다. 행을 거꾸로 도는 루프와 가변 인덱스를 고정된 5 비트 우선순위 선택으로 바꿨다.
- 충돌 검사와 외계인 그리기. 외계인 25 마리마다 하던 비교를, 무리의 기준 좌표에서 잰 상대 좌표 하나와 상수 범위 비교로 바꿨다. 원래 RTL 은 7 비트와 6 비트로 계산하면서 값이 넘치는데, 넘침 동작도 그대로 맞췄다. group_x 는 2~70, cannon_x 는 1~118 범위를 벗어나지 않는다는 것을 RTL 에서 확인하고 그 범위 안에서 같은 결과가 나오게 했다. 화면 아래로 넘어간 외계인 줄은 원래 RTL 처럼 그리지 않는다.
- 리셋. START 상태에서 매번 값이 정해지거나 쓰이기 전에 반드시 값이 들어가는 레지스터 80 개는 비동기 리셋을 뺐다. 그러면 DFFSR(2,592 μm^2) 대신 DFFPOSX1(1,296 μm^2)이 쓰인다.
- 합성 매핑. qflow 의 abc_script 변수로 면적 위주 매핑(amap)을 썼다. 같은 RTL 을 수업 기본 스크립트로 합성하면 0.812 mm^2 라서 이것까지 해야 들어갔다.

TABLE 3. 기준 RTL 과 면적 최적화 RTL

	기준 No_Score	면적 최적화
셀 수 (합성)	1,829 (DFFSR 102)	1,170 (DFFSR 23, DFFPOSX1 80)
셀 실면적	1.083 mm^2	0.673 mm^2 (-38%)
레이아웃	1344.6 × 1057.05 µm	936.6 × 913.05 µm (밀도 0.85)
가장 긴 경로 (배선 지연 제외)	6.1 ns	9.9 ns
FPGA (EP4CE6)		441 LE, Fmax 104 MHz

(a) 기준 RTL, 1344.6 x 1057.05 um



(b) 면적 최적화 RTL, 936.6 x 913.05 um

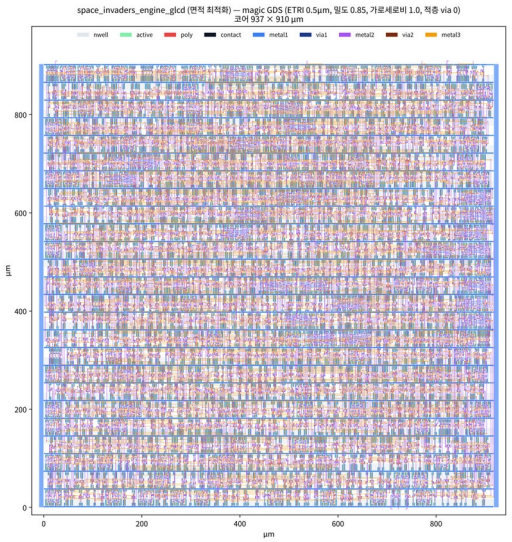


Fig. 2. 두 레이아웃을 GDS 파일에서 그린 그림. 그림마다 자기 크기에 맞춰 그려서 축척은 다르다. 크기는 위에 적은 값이다.

바꾼 RTL 이 원래 RTL 과 같은지는 두 RTL 을 한 Verilator 모델에 넣고 같은 입력을 준 뒤 매 클럭 v_sync 와 lcd_data 를 비교해서 확인했다. 입력은 사람이 누르듯 버튼을 눌렀다 떼는 무작위 패턴에 가끔 리셋을 섞었다. 짝수 시드에서는 일부러 외계인 미사일 밑으로 따라다니게 해서 피격과 GAME OVER 가 자주 나오게 했다. 시드 8 개, 4 억 클럭씩(약 39 만 프레임)에서 피격 656 번, GAME OVER 199 번을 거쳤고 불일치는 0 이었다. 배선 뒤 게이트 넷리스트도 같은 방법으로 1.2 억 클럭을 돌려 불일치가 없었다. 이때 리셋 없는 플립플롭의 처음 값은 무작위로 두었다.

이 비교 방법이 차이를 실제로 잡는지도 확인했다. 같은 방법으로 원래 RTL 과 v_3 을 비교하니 첫 시드에서 바로 불일치가 나왔다. v_3 의 충돌 검사는 최대 25 클럭 늦게 반응하기 때문이다. 2 절의 2,400 프레임 시험에서는 이 차이가 화면에 드러나지 않았을 뿐이다. 화면이 같다는 것과 회로가 같다는 것은 다른 이야기였다.

처음에는 이 비교가 실패했다. 따라가 보니 원래 RTL 의 버그였다. 발사 위치를 고르는 invader_alive[(shoot_row * 3'd5) + shoot_col]에서 인덱스 식이 3 비트로 계산되어 8 로 나눈 나머지가 된다. 그래서 2 열 외계인(22, 17, 12, 7, 2 번)이 아니라 6, 1, 4, 7, 2 번 비트를 본다. 2 열 외계인이 죽어도 그 자리에서 미사일이 나올 수 있다. 대화 기록에서 인공지능이 "명시적 비트 유도 배정으로 원천 교정했다"고 한 바로 그 식이다. 동작을 맞추려고 면적 최적화 RTL 도 이 동작을 그대로 따르게 했고, 고치려면 한 줄만 바꾸면 된다.

대가도 있다. 면적 위주 매핑을 쓰니 가장 긴 경로가 6.1 ns 에서 9.9 ns 로 늘었다. 프로토타이핑 클럭 100 kHz 나 예제 주석의 50 MHz 에는 문제가 없다. 배선에서 계산한 클럭 스큐를 넣은 홀드 검사도 모든 끝점이 양수였다.

5. 과제 19: 적층 비아 제거

키트 기본 설정의 레이아웃에서 make stack 은 적층 비아 4 개를 찾았다(블로그 예시는 2 개). 블로그는 magic 에서 손으로 고치는 방법을 안내한다. 나는 배선 DEF 에서 via1 바로 위에 via2 가 있는 자리를 찾고, 그 자리에 1.2 × 1.2 µm metal3 장애물을 넣은 뒤 다시 배선하는 것을 반복했다. 손으로 고치면 LVS 와 DRC 를 다시 해야 하고 위치가 많으면 실수하기 쉬운데, 이 방법은 배선기가 알아서 다른 길로 돌아간다. 기준 레이아웃은 5 개에서 1 개, 0 개로 두 번 만에 끝났고, 면적 최적화 레이아웃은 세 번 만에 0 개가 됐다. 밀도 0.9 레이아웃에서는 여덟 번을 돌려도 1~2 개가 계속 새로 생겨서 밀도 0.85 를 최종으로 골랐다.

TABLE 4. 최종 레이아웃 검사 결과 (두 레이아웃 공통)

검사	결과
키트 check_via_stack.py (make stack)	No stacked Contact and/or Via found
키트 run_drc.sh (make drc)	Total DRC errors found: 0
qflow LVS (netgen)	Circuits match uniquely
키트 run_lvs.sh (make lvs)	Circuits match uniquely

6. 과제 16·17: PSCE 키트

6.1 에뮬레이션

피코에 PSCE-MI 펌웨어를 올리고 FPGA 에 래퍼를 포함한 면적 최적화 RTL(455 LE)을 올린 뒤, 시뮬레이션 때의 SystemC 테스트벤치를 그대로 빌드해 돌렸다. Connection established 가 나온 뒤 5 분 동안 133 프레임이 돌았고, 한 화면에 약 2.17 초가 걸렸다. 8,192 클럭에 2.17 초이니 초당 약 3,800 클럭이고, 클럭 에지 하나마다 USB 를 한 번 왕복하는 데 약 130 µs 가 드는 셈이다. 같은 RTL 을 PC 에서 시뮬레이션하면 훨씬 빠르다. 블로그에서 에뮬레이션이 시뮬레이션보다 느리다고 한 이유가 이 숫자에서 드러난다.

캡처한 화면을 RTL 에서 기대되는 화면과 화소 단위로 비교하니, 정확히 1 화소 밀었을 때 8,192 화소 중 8,191 개가 맞았다. 시뮬레이션에서 본 1 화소 밀림이 FPGA 에서도 그대로 나온 것이다. 맞지 않은 1 개는 맨 오른쪽 아래 화소로, 이 GLCD 모델이 매 화면 한 번도 그리지 않는 자리다.

6.2 프로토타이핑

프로토타이핑 펌웨어와 DUT 만 있는 FPGA 이미지(441 LE)를 올렸다. 피코가 PWM 으로 100 kHz 클럭을 GPIO28 로 FPGA 에 주고, 게임 화면은 키트의 OLED 에 나온다. OLED 에는 한 화소씩 번갈아 켜진 바둑판 무늬, 곧 START 화면이 나왔다(Fig. 3). 피코의 클럭으로 FPGA 안의 게임이 돌고, 피코가 화면 데이터를 읽어 OLED 에 그리는 경로가 동작한다.

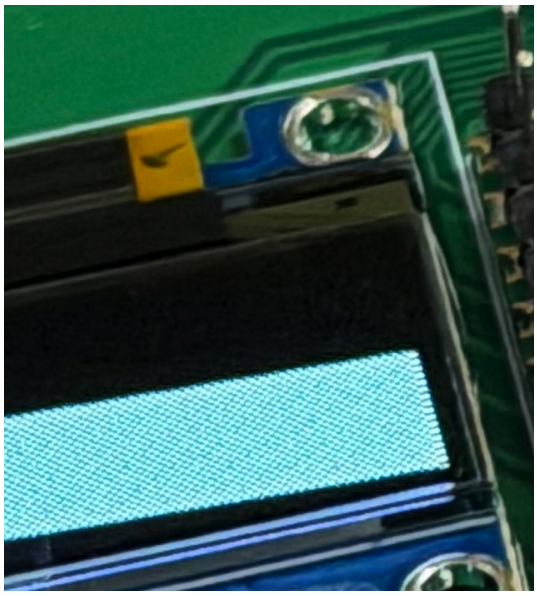


Fig. 3. 프로토타이핑 중인 키트의 OLED. 바둑판 무늬의 START 화면이다(휴대폰 사진을 잘라 늘림).

응용 펌웨어(space_invaders_SA.ino)는 인터럽트 두 개로 화면을 받는다. clk_o(DUT 가 내는 반전 클럭)의 상승 에지마다 lcd_data 를 읽어 128 × 64 비트맵을 채우고, v_sync 상승 에지에서 화소 카운터를 0 으로 돌리고 그리기 플래그를 세운다. 그리기는 두 번째 코어(loop1)가 I2C 로 OLED 에 한다. 100 kHz 이면 인터럽트가 10 µs 마다 오고, 한 화면(8,192 클럭)은 약 82 ms, 게임 틱(32,768 클럭)은 약 0.33 초다. lcd_data 는 클럭 상승 에지에서 바뀌므로 clk_o 상승 에지, 곧 클럭 하강 에지에서 읽으면 값이 안정되어 있다. 반전 클럭을 따로 내보낸 이유가 이것으로 보인다.

버튼은 PC 쪽 SystemC 응용이 SDL 키 이벤트를 받아 매 클럭 1 바이트(bit3 rst_n, bit2 left, bit1 right, bit0 fire)로 시리얼 전송한다. 피코의 loop()가 이것을 받아 GPIO 로 내보낸다. 이 입력은 FPGA 클럭과 동기가 아니고, RTL 에도 동기화용 플립플롭이 없다. 좌우 이동은 RTL 이 한 화면에 한 번만 읽어서 1 화소씩 움직이므로, 키를 82 ms 이상 누르고 있어야 반응한다. 코드에서 눈에 띈 점도 두 가지 있다. 두 코어가 같이 쓰는 bUpdateBuffer 가 volatile 이 아니고, 비트맵이 한 벌뿐이라 OLED 에 보내는 동안 다음 화면이 그 위에 덮어쓴다.

2 부. 4 차 MPW 설계 계획서: bb_node

7. 개요

bb_node 는 배터리 없는 백스캐터 센서 노드의 디지털 베이스밴드 칩이다. 코어 레이아웃과 28 핀 칩 탑 GDS 까지 만들었고, 코어 DRC 0, LVS 일치, 셋업과 홀드 만족을 확인했다.

TABLE 5. bb_node 개요

항목	내용
설계자	안준영 (2024-11078, 기계공학부)
칩 이름	bb_node v3
공정	ETRI 0.5 µm CMOS (2P3M), 2026 년 4 차 MPW
패키지와 패드	28 핀 SOP, 수업 패드 프레임 MPW_PAD_28Pin_IO_Games
설계 영역	코어 984 × 951 µm, 표준 셀 1,350 개
동작 클럭	32.768 kHz (시험할 때는 피코가 1 kHz~4 MHz 로 바꿔 줄 수 있음)
권장 전원	3.3 V (피코와 FPGA 로 시험할 때의 기준, 12 절)

8. 필요성과 목표

배터리 없는 백스캐터 노드는 센서 읽기, 패킷 만들기, 반사 부호화, 리더 명령 듣기를 끊임없이 해야 한다. 연구용 시제품은 이 일을 범용 MCU 로 한다. 예를 들어 수중 압전 백스캐터 PAB(SIGCOMM 2019)는 MSP430 을 쓴다. 이 칩은 이 네 가지 일을 고정 기능 회로로 옮긴다.

MCU 를 빼면 펌웨어가 하던 판단을 회로가 스스로 해야 한다. 그래서 목표를 다음 세 가지로 잡았다.

- 노드 클럭이 ±50% 틀어져도 리더 명령을 받는다. 리더가 보내는 기준 심벌 길이의 절반을 매번 0 과 1 의 판정 기준으로 쓴다.
- 반송파가 있을 때만 반사하고, 리더가 말을 시작하면 송신을 멈춘다.
- 리더가 패킷 손실을 구분할 수 있게 순번과 miss, err 플래그를 패킷에 싣는다.

0.5 µm 5 V 공정이라 µW 급 전력은 목표로 삼지 않는다. 대신 전력을 줄이는 구조(32 kHz 클럭, 센서 전원 게이팅, 반송파 대기, 이벤트 구동, 휴면)가 실리콘에서 제대로 동작하는지 보인다. 압전 소자, 정류기, 포락선 검출기, 반사 스위치는 칩 밖에 둔다.

9. 사양

신호 12 핀(입력 7, 출력 5)과 전원·접지 16 핀을 쓴다. CLK 와 RESET 은 패드 프레임의 원래 자리인 22 번과 23 번을 그대로 쓴다.

TABLE 6. 핀 배치

핀	신호	방향	패드	설명
22	CLK	입력	PIC	기준 클럭 32.768 kHz
23	RST_N	입력	PIC	동기 리셋, Low 활성
24	TRIG	입력	PIC	외부 이벤트 (상승 에지)
25	SPI_MISO	입력	PIC	센서 ADC 데이터
26	DL_IN	입력	PIC	하향 OOK 포락선, High 면 반송파 있음
27, 28	MODE0, MODE1	입력	PIC	00 일반, 01 명령 대기, 10 연속 송신, 11 시험 톤
9	BS_OUT	출력	POB8	반사 스위치 구동 (FM0)
10, 11	SPI_SCLK, SPI_CSN	출력	POB8	센서 SPI (CLK/2, mode 0)
12	SENS_EN	출력	POB8	센서 전원 게이팅
13	STATUS	출력	POB8	활동 중 표시 (듀티 사이클 측정용)
1~8	VDD	전원	PVDD	칩 전원
14~21	GND	접지	PVSS	칩 접지

TABLE 7. 주요 동작 (32.768 kHz 기준)

항목	값
상향 패킷	53 비트: PILOT 4, 프리앰블 0x7E, ID, STAT, DATA 16, CRC-8
상향 부호	FM0, 서브캐리어 선택 가능. 기본 2.048 kbps, 패킷 하나에 25.9 ms
하향 프레임	24 비트: ID 8, OP 4, ARG 8, CRC-4. 펄스 폭 부호, 기준 심벌로 자기 보정
명령	SET_ID, SET_PERIOD, SET_HB, SET_CFG, QUERY, SET_WAKE, GET_CFG, CLR
측정 주기	125 ms × PERIOD (기본 0.5 s, 0 이면 휴면)
반송파 감지	DL_IN 이 128 클럭(3.9 ms) 이상 High
센서 예열	WAKE × 8 클럭 (기본 0.49 ms)

10. 구조

RTL 은 최상위 bb_node 와 하향 복원기 bb_dl 두 모듈이다. 합성하면 표준 셀 1,187 개(플립플롭 173 개)가 되고, 배치 뒤 팬아웃 버퍼가 들어가 레이아웃에는 1,350 개가 들어간다.

TABLE 8. 블록 구성

블록	하는 일
하향 복원기 (bb_dl)	2 단 동기화, 3 표본 다수결 필터, 구분자와 기준 심벌 측정, 24 비트 수신, CRC-4
명령 처리	ID 비교, 레지스터(ID, PERIOD, HB, WAKE, CFG) 갱신, 응답 요청
주기 타이머	4,096 클럭 tick, PERIOD 비교
측정·송신 상태기계	IDLE → WAKE → SPI → WAIT(반송파) → TX, 요청 우선순위
SPI 마스터	16 비트 읽기, CLK/2
패킷·FM0 송신기	비트 선택, CRC-8, FM0 레벨, 서브캐리어
상태 표시	SEQ, miss, err, STATUS 핀

면적은 기능을 빼지 않고 카운터를 나눠 쓰는 방식으로 줄였다. 첫 합성의 1,222 셀에서 1,187 셀로 3% 줄었다.

• 반 비트 카운터 하나로 반 비트 시간, 센서 예열, 시험 톤을 모두 센다.

• SPI 비트 카운터를 송신 비트 번호로 다시 쓴다.

• 시험 톤은 FM0 레벨 플립플롭을 같이 쓰고, 기준 심벌의 절반은 시프트로 구한다.

11. 설계 흐름과 진행 현황

RTL 부터 칩 탑 GDS 까지 만들었고, 남은 단계는 FPGA 보드 시험과 제출이다. 모든 단계는 스크립트 한 번으로 다시 만들 수 있다.

TABLE 9. 설계 단계

단계	도구	결과	상태
C++ 기준 모델	g++	패킷, CRC, FM0, 리더 복호기, 하향 해석	완료
RTL 시뮬레이션	Verilator 5.030, SystemC 3.0.2	시나리오 16 개, 확인 항목 111 개 PASS	완료
합성	yosys (qflow), ETRI 셀	1,187 셀, 플립플롭 173 개	완료
배치	graywolf	26 행, 밀도 0.90, 핀을 패드 단자 위치에 고정	완료
배선	qrouter 1.4.90	금속 3 층, 실패 0, 적층 비아 0	완료
코어 DRC·LVS	magic 8.3, netgen 1.5	DRC 0 (키트 설정), LVS 일치	완료
칩 탑	키트 스크립트 (xPad, xPin_Route_Metal, generate_chip)	bb_node_Top.gds, 연결 26 개 확인	완료
STA	vesta, 배선으로 계산한 클럭 스큐	셋업·홀드 만족	완료
FPGA 에뮬레이션	PSCE 키트, Quartus	래퍼와 루프백 시험 111 개 PASS	11 월 20 일 보드 시험
제출	moafab.kr	설계 DB	12 월 7 일까지

코어 배선에서는 두 가지를 따로 맞췄다. 첫째, via1 바로 위에 via2 를 쌓을 수 없어서, 1 부 5 절과 같은 방법으로 적층 비아 자리에 metal3 장애물을 넣고 다시 배선했다(처음 6 개에서 0 개). 둘째, 코어 핀을 패드 단자와 같은 x 위치에 두어 패드까지 가는 배선이 직선이 되게 했다.

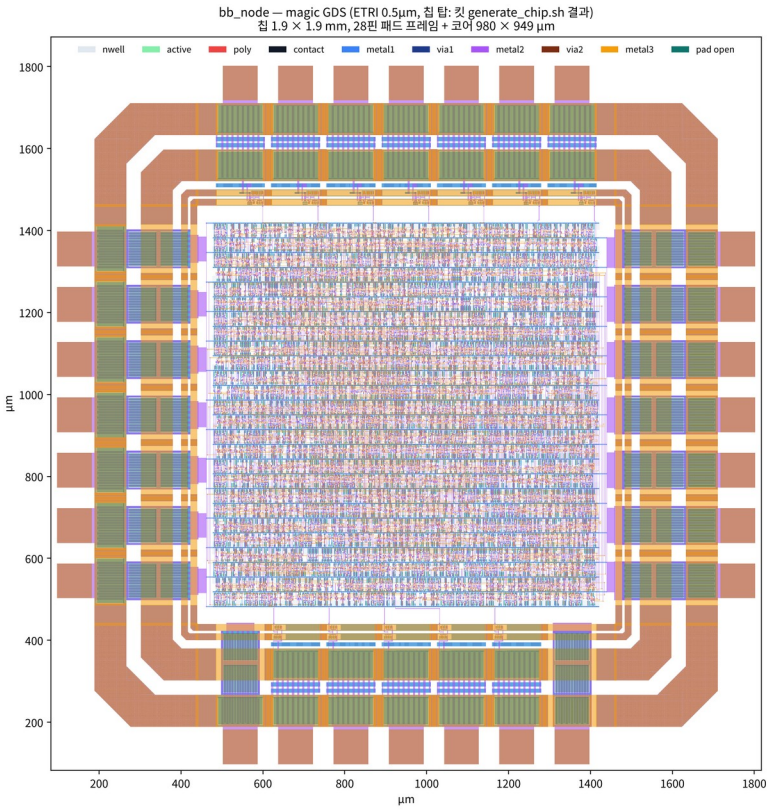


Fig. 4. bb_node 칩 전체를 제출용 bb_node_Top.gds 에서 그린 그림. 가운데가 코어, 위쪽 7 개가 입력 패드, 아래쪽 5 개가 출력 패드, 왼쪽과 오른쪽이 VDD 와 GND 패드다.

12. 검증 계획과 결과

C++ 기준 모델이 기대 출력을 만들고, RTL 과 게이트 넷리스트를 같은 테스트벤치로 이 출력과 비교한다. FPGA 에뮬레이션도 같은 테스트벤치를 쓴다. 지금까지 한 검증은 TABLE 10 과 같이 모두 통과했다.

TABLE 10. 검증 항목과 결과

검증	방법	결과
기능 (RTL)	송신 파형을 클럭마다 기준 모델과 비교, 비트 속도를 모르는 리더 복호기로 해독	111 개 PASS
테스트벤치 강도	버그 12 개를 일부러 넣음 (돌연변이 시험)	12 개 모두 검출
클럭 오차	리더 시간 배율 0.5~2.0	명령 모두 수신 (고정 기준은 ±25%에서 실패)
게이트 수준	배선 DEF 에서 다시 뽑은 넷리스트, Liberty 에서 자동 생성한 셀 모델	111 개 PASS, 출력이 RTL 과 같음
DRC	magic, 키트 설정(scalegrid 1 3, drc euclidean on), 키트 run_drc.sh	코어 0
적층 비아	키트 check_via_stack.py	0
LVS	netgen	소자 1,350 개, 넷 1,359 개 일치
칩 탑	GDS 에서 연결 26 개 기하 검사, 칩 전체 DRC 를 패드 프레임 단독 결과와 비교	연결 실패 0, 패드 셀 밖 오류 0
타이밍	vesta, 배선으로 계산한 클럭 스큐(Elmore)	셋업 여유 25 µs, 홀드 여유 최소 40 ps
입력 패드	ngspice, 키트 BSIM3 모델	VDD 3.3 V 에서 문턱 1.50 V, VIH 1.70 V
FPGA 연결	PSCE 트랜잭션 계층을 루프백으로 시뮬레이션	111 개 PASS
리더 펌웨어	기준 모델 대조, RTL 파형 복호	패킷 1,485 개와 명령 2,000 개 일치, 파형 패킷 89 개 CRC 통과

입력 패드를 시뮬레이션해 보니, 칩을 5 V 로 돌리면 3.3 V 입력 하나마다 약 340 µA 가 관통해 흐른다. 그래서 피코와 FPGA 로 시험할 때는 칩 VDD 를 3.3 V 로 한다.

13. 측정 계획

칩이 오기 전에는 같은 RTL 을 키트 FPGA 에 올려 시험 환경을 먼저 검증한다. 칩이 오면 같은 시험을 SOP28 소켓에 꽂은 칩으로 반복한다. 칩 VDD 는 3.3 V 로 두고, 피코는 GP2~GP13 열두 핀으로 칩의 신호 핀 12 개에 연결한다.

TABLE 11. 측정 단계

단계	구성	확인할 것
1. PSCE 에뮬레이션	PC 의 SystemC 테스트벤치 → 피코 PSCE-MI → Cyclone IV EP4CE6	시뮬레이션과 같은 확인 항목 111 개 통과
2. 피코 리더 시험 (FPGA)	피코가 클럭원, 리더, 센서, 시험기 역할. bringup 자동 시험	PIO 타이밍, 리셋 상태, 설정 읽기, 반송파 게이팅, 주기
3. 칩 기본 동작	칩, SOP28 소켓, 피코, 3.3 V	2 단계와 같은 bringup 항목 7 개
4. 하향 강건성	리더 시간 배율 0.5~2.0, 칩 클럭 주파수 변화	명령 오류율
5. 전류	주기, 비트 속도, 모드별 공급 전류와 STATUS 듀티	패킷당 활동 에너지와 대기 전류
6. 음향 시험 (가능하면)	BS_OUT 으로 압전 반사 스위치 구동, 하이드로폰으로 복호	실제 채널에서 패킷 수신

14. 일정

최종 GDS 는 11 월 25 일까지 확정하고, 12 월 7 일 마감 전에 제출한다. 칩은 설계 DB 를 제출하고 약 5 개월 뒤에 받는다.

TABLE 12. 일정

날짜	할 일
2026-11-13	제출 형식과 시험 보드 전원 조건 확인, Quartus 로 FPGA 래퍼 합성
2026-11-20	FPGA 에서 PSCE 시험과 피코 bringup 시험
2026-11-25	최종 GDS 확정 (필요하면 KLayout DRC 덱 실행)
2026-12-07	moafab.kr 신청과 설계 DB 제출, 개인 결과 보고서
2027 년 5 월 무렵	칩 수령, 13 절의 3~6 단계 측정

15. 위험과 대응

가장 큰 위험은 제출 형식이나 시험 보드 조건이 지금 가정과 다른 경우다. 두 경우 모두 스크립트로 다시 만들 수 있어 하루 안에 대응할 수 있다.

TABLE 13. 위험과 대응

위험	영향	대응
제출 패키지 형식이나 파일 이름이 다름	제출 지연	키트 형식(Makefile, bb_node_Top.gds)을 따랐고, 제출 전에 형식을 확인
시험 보드가 5 V 고정	피코 직결 불가, 입력마다 관통 전류	출력 5 개에 레벨 변환, 또는 따로 만든 3.3 V 소켓 보드
패드 프레임 핀 배치 변경 요구	코어 핀 위치와 배선을 다시 해야 함	set_pins.py 와 build_chip_top.sh 로 다시 생성
KLayout DRC 덱 결과가 magic 과 다름	제출 전 수정 필요	제출 전에 덱을 돌려 차이를 확인

남은 의문

- 면적 위주 매핑은 가장 긴 경로를 60% 넘게 늘렸다. 칩을 시험할 클럭이 정해져 있으면 면적과 지연 중 무엇을 먼저 맞출지 처음부터 정할 수 있다. bb_node 는 32.768 kHz 라 면적을 먼저 맞췄다.
- 리셋을 뺀 레지스터는 칩에 전원이 들어온 직후 아무 값이나 가진다. 이 게임에서는 START 상태가 모두 덮어써서 출력이 같지만, 전원이 들어온 직후의 동작을 따지는 검사(X 전파 시뮬레이션 등)를 표준 흐름에 넣어야 하지 않을까.
- 버튼 입력은 FPGA 클럭과 동기가 아니다. FPGA 에서는 문제가 없어 보여도, 칩으로 만들면 준안정 상태가 생길 수 있다. 입력에 플립플롭 두 개를 두는 비용은 면적의 1%도 안 된다. bb_node 는 비동기 입력인 DL_IN 에 2 단 동기화를 넣었다.

- 적층 비아를 장애물로 피하는 방법은 밀도가 높으면 수렴하지 않았다. 배선기 설정에서 적층을 막았는데도 생기는 이유는 아직 모른다.

마치며

이번 주에 가장 오래 걸린 일은 회로를 줄이는 것보다, 줄인 회로가 원래 회로와 같다는 것을 보이는 일이었다. 2,400 프레임의 화면이 같아도 회로는 달랐고, 매 클럭을 비교하고서야 원래 RTL 의 인덱스 버그와 v_3 의 늦은 충돌 검사가 드러났다. 인공지능은 면적을 80% 줄였다고 했지만 실제로는 늘었고, 고쳤다는 식에 버그가 남아 있었다. 다음에는 2 부의 일정대로 bb_node 를 보드에서 시험하고 4 차 MPW 에 낸다.